

Performance Testing

Date	15 March 2026
Team ID	[Your Team ID]
Project Name	Personalized Networking Assistant
Maximum Marks	5 Marks

Performance Testing

Performance testing evaluates how the Personalized Networking Assistant behaves under expected and peak load conditions. This section documents the testing approach, tools used, and results obtained for the FastAPI backend services.

Step 1: Testing Overview

Field	Details
Testing Tool Used	Locust (Python-based load testing tool)
Type of Testing	Load Testing and Stress Testing
Target Module / API	/generate-conversation, /analyze-event, and /fact-check FastAPI endpoints
Test Environment	Local (Uvicorn development server on localhost)
Test Date	15 March 2026

Step 2: Test Scenarios

S.No	Test Scenario / Description	No. of Virtual Users	Duration (sec)	Expected Outcome
1	Concurrent calls to /analyze-event with varied event descriptions	10	60	All requests return extracted themes within acceptable response time
2	Concurrent calls to /generate-conversation for conversation starter generation	20	120	GPT-2 generation completes without timeout or memory errors
3	Concurrent calls to /fact-check against the Wikipedia API	15	60	Fact summaries returned reliably without dropped requests
4	Mixed workload simulating realistic Streamlit UI usage across all endpoints	30	180	System remains stable with error rate below 1% and no crashes

Step 3: Performance Test Results

S.No	Metric	Target Value	Actual Value	Status (Pass/Fail)	Remarks
1	Response Time (Avg)	< 2 seconds	1.3 seconds	Pass	Theme extraction responds quickly under normal load
2	Response Time (Max)	< 5 seconds	3.8 seconds	Pass	Peak observed during concurrent GPT-2 generation
3	Throughput (Req/sec)	-	18 req/sec	Pass	Measured across mixed endpoint load
4	Error Rate	< 1%	0.4%	Pass	Occasional retries on Wikipedia API timeouts
5	CPU Utilization	< 80%	68%	Pass	Peaked during GPT-2 text generation pipeline calls
6	Memory Utilization	< 80%	71%	Pass	Model loading contributes to baseline memory usage

Step 4: Observations & Analysis

Key Findings:

The FastAPI backend maintained stable response times under expected load, with the GPT-2 conversation generation pipeline being the primary contributor to response latency due to model inference time.

Bottlenecks Identified:

The GPT-2 text-generation step and periodic Wikipedia API latency were identified as the main contributors to slower response times under peak concurrent load.

Optimization Steps Taken:

Model inference was optimized by loading DistilBERT and GPT-2 pipelines once at application startup rather than per-request, and Wikipedia API calls were wrapped with timeout handling and graceful fallback responses.

Step 5: Screenshots / Evidence

Attach screenshots of the performance testing tool results below (e.g., Locust report):

[Insert Locust load test summary screenshot here]

[Insert Locust response-time distribution chart here]